Computer Science Department

Master Degree in Computer Science

PROJECT DOCUMENTATION for BIG DATA

Evaluating Temporal Link Prediction methods to generate synthetic social networks data

Author:

Mattia Curri

ACADEMIC YEAR **2025-2026**

Contents

1 Business Understanding **1**
- 1.1 Determine Business Objectives . . . 1
- 1.2 Assess Situation . . . 2
- 1.3 Determine Data Mining Goals . . . 3

2 Data Understanding **4**
- 2.1 Collect Initial Data . . . 4
- 2.2 Describe Data . . . 4
- 2.3 Explore Data . . . 5

3 Data Preparation **10**
- 3.1 Select Data . . . 10
- 3.2 Clean Data . . . 10
- 3.3 Construct Data . . . 11
- 3.4 Integrate Data . . . 12
- 3.5 Format Data . . . 12

4 Modeling **13**
- 4.1 Select Modeling Techniques . . . 13
- 4.2 Generate Test Design . . . 17

5 Evaluation **21**
- 5.1 Assess Model . . . 21

1 Business Understanding

This chapter covers the business understanding phase of the CRISP-DM methodology.

1.1 Determine Business Objectives

Background

Social network analysis (SNA) has emerged as a critical pursuit in understanding the complex dynamics of modern online communities. While traditional statistical techniques remain valuable, SNA introduces specialized measures to capture the dependencies between social entities, characterize their behaviors, and assess their impact on the network both structurally and temporally [1]. The fundamental principle of SNA lies in shifting the analytical focus from individual social actors to the relationships and interactions that bind them, revealing patterns that emerge from the network's underlying structure rather than from isolated entities [1].

In the globalized digital landscape, social networks have become one of the favorite means for modern society to perform social interactions and exchange information via the Internet, allowing people to share their contents without boundaries [2, 3]. Through these platforms, individuals communicate and express their comments, likes, and interests in a fast and easy manner, creating complex connections that can be immediately visualized as large graphs [2]. This massive spread of social media has introduced unprecedented opportunities for information dissemination [4]. However, this very interconnectedness presents significant challenges: the same platforms that facilitate legitimate social connections also provide powerful tools for malicious activities, including misinformation campaigns, cyberbullying, hate speech propagation, and the radicalization of vulnerable individuals [5, 3, 4, 6].

Risky users exploit the spreading power of social networks for illegal or harmful activities such as inciting religious or political extremism, disseminating disinformation, or recruiting. For these reasons, detecting them has become a fundamental task for ensuring the safe use of these platforms [3, 5, 6]. This challenge is particularly acute when dealing with borderline users who appear risky from certain perspectives but not from others, and is further complicated by the dynamic nature of social networks [3, 5]. User relationships, content semantics, and behavioral patterns continuously evolve; models that treat networks as static entities fail to capture users who transition from safe to radicalized behavior over time [5].

The unstructured nature of user interactions on social media platforms renders manual identification of malicious actors both tedious and impractical at scale [4]. Recent research has increasingly focused on leveraging graph-based representations, where users constitute nodes and their interactions form edges, to address this complexity [4, 7]. Link prediction, defined as the process of predicting future connections between pairs of unconnected links based on existing connections [2], represents a fundamental problem in network analysis that has evolved from static snapshot-based approaches to temporal methodologies that exploit multiple network states across time to predict future or re-occurring

connections [1, 2]. This temporal perspective aligns with the recognition that real-world networks are not static but dynamic and continuously evolving, requiring analytical frameworks that can capture growth patterns and temporal dependencies [1].

Given the urgency of protecting society from exposure to discriminatory, hateful, and violent content at the massive scale of contemporary social networks, there is a critical need for effective automated systems to assist Law Enforcement Agencies in identifying and addressing users who disseminate malicious content [6]. This work addresses this need by developing methods that integrate multiple perspectives of user behavior, account for the temporal evolution of networks, and leverage the relational structure inherent in social media platforms to enable accurate and timely identification of risky users.

Business Goals

The primary business goal of this project is to develop an effective temporal link prediction system for social networks that can:

- Predict future connections between users based on their historical interactions and content
- Evaluate the effectiveness of incremental learning approaches for temporal network evolution
- Given a dataset of users and relative posts, generate realistic synthetic social network structures
- Provide insights into how network properties evolve over time

Business Success Criteria

The project will be considered successful if it achieves the following outcomes:

- Development of a temporal link prediction model based on EvolveGCN
- Quantitative comparison of model performance using standard metrics (Mean Average Precision, AUC, Precision, Recall, F1)
- Successful transfer learning application: trained models can generate plausible network structures on synthetic datasets
- Analysis of network evolution patterns and structural properties across temporal snapshots
- Generation of a large synthetic network of 1000 artificial users with realistic connection patterns

1.2 Assess Situation

Inventory of Resources

The available data are being collected from the social network platform Gab[1], known for its minimal content moderation policies and popularity among far-right communities, over a defined time period.

Current resources available for the project are the following machines:

1. CPU: AMD Ryzen 7 7700, 8 cores, 16 threads; GPU: NVIDIA RTX 5060Ti, 16 GB VRAM; RAM: 48 GB
2. CPU: AMD Ryzen 7 7840HS, 8 cores, 16 threads; GPU: Integrated AMD Radeon 780M; RAM: 48 GB

[1] https://gab.com/

3. CPU: Intel Core i7-10750H, 6 cores, 12 threads; GPU: NVIDIA GTX 1650 Mobile Max-Q, 4 GB VRAM; RAM: 16 GB

Identify Constraints

There are no significant constraints anticipated for this project. However, potential challenges include:

- Data Quality: The collected data may contain noise, missing values, or inconsistencies that could affect model performance.
- Computational Resources: Training complex models on large datasets may require significant computational power and time.
- Ethical Considerations: Handling user data requires adherence to privacy regulations and ethical guidelines to ensure responsible use of information.

Assumptions

- The dataset from Gab is representative of broader social network dynamics and user behaviors.
- A connection between users exists if they are connected in one snapshot
- The graph is directed and unweighted
- If a connection exists between two users in a snapshot, it will also exist in all subsequent snapshots

1.3 Determine Data Mining Goals

Data Mining Goals

Here we translate the business goals into specific data mining objectives:

- Develop a temporal link prediction model using EvolveGCN to forecast future user connections based on historical interaction data.
- Evaluate the model's performance using metrics such as Mean Average Precision (MAP), Area Under the Curve (AUC), Precision, Recall, and F1-score.
- Adapt and test the trained model for generating synthetic social network structures.
- Analyze the generated network properties, including degree distribution, clustering coefficient, and average path length.

Success Criteria

The success of the data mining efforts will be measured by:

- Achieving high performance on link prediction tasks
- Demonstrating the model's ability to generalize to synthetic datasets, producing realistic network structures.
- Providing comprehensive insights about the generated network's structural properties.

2 📖 Data Understanding

This chapter covers the data understanding phase of the CRISP-DM methodology.

2.1 Collect Initial Data

The available data are being collected from the social network platform Gab[1]. The dataset includes several snapshots, in particular, 2016-2021, 2022, 2023, 2024, January-July 2025 and July 2025. For every snapshot, there three files:

- posts_current_snapshot.csv: contains all the posts made by users in the platform for that snapshot;
- posts_incremental.csv: contains all the posts made by users in the platform since the previous snapshots up to the current one;
- social_network.edg: contains the social network graph in edge list format, where every line represents a directed edge from a follower to a followee (X follows Y).

Along with them, there are also 3 synthetic batches for the subsequent task of using the trained model on the original data to generate links.

For both the datasets, the embedding has been provided, computed using a BERT model.

2.2 Describe Data

Raw data files are in CSV format, with the exception of the social network graph which is in edge list format. For each post, the following attributes are available:

- unnamed_0 (autoincrement): index of the post, autoincremented
- id (integer): unique identifier of the post
- content (string): text of the post
- account_id (integer): unique identifier of the user who made the post
- created_at (string): ISO 8601 timestamp of when the post was created
- language (string): language code of the post
- replies_count (integer): number of replies to the post
- reblogs_count (integer): number of reblogs of the post

[1] https://gab.com/

- reactions_counts (string): JSON dictionary of reactions to the post
- quotes_count (integer): number of quotes of the post
- timestamp (string): ISO 8601 timestamp of the post (created_at + timezone)

2.3 Explore Data

Data quality verification examines completeness, consistency, accuracy, and potential issues in the dataset.

Completeness Analysis

Missing Values

The dataset shows excellent completeness for most attributes. The only attribute with missing values is language, with 877 missing values (0.48%) in the aggregated current snapshots and 3,747 (0.85%) in the incremental data. All other attributes are 100% complete.

Table 2.1: Missing values summary (aggregated current snapshots)

Attribute	Missing Count	Missing %
language	877	0.48%
All other attributes	0	0.00%

The missing language values likely correspond to posts where automatic language detection failed, possibly due to very short content, mixed-language posts, or non-standard text.

No Empty Content

Importantly, no posts with empty content fields were found, ensuring all records contain textual data for analysis.

Consistency Checks

Snapshot vs. Incremental Consistency

A critical quality check verified that each current snapshot is fully contained within its corresponding incremental file. All six snapshots achieved 100% coverage:

Table 2.2: Snapshot-incremental consistency verification

Snapshot	Snapshot Posts	Incremental Posts	Coverage
2016-2021	18,926	18,926	100%
2022	13,108	32,034	100%
2023	13,376	45,410	100%
2024	13,400	58,810	100%
Jan-Jul 25	44,878	103,688	100%
Jul 25	78,857	182,545	100%

Temporal Consistency

The created_at and timestamp fields are identical across all snapshots, confirming consistent timestamp handling. No posts with future dates were detected.

Unique Identifiers

No duplicate post IDs were found, either within individual snapshots or across the aggregated dataset. Each post maintains a unique identifier throughout the dataset.

Duplicate Content Analysis

While post IDs are unique, content-level duplicates exist in the dataset. Table 2.3 summarizes the duplicate content findings.

Table 2.3: Duplicate content analysis per snapshot

Snapshot	Duplicate Posts	Percentage
2016-2021	339	1.79%
2022	396	3.02%
2023	563	4.21%
2024	563	4.20%
Jan-Jul 25	3,377	7.52%
Jul 25	7,919	10.04%

The increasing duplicate rate over time suggests growing repetitive posting behavior, possibly including:

- Users reposting the same content multiple times;
- Common phrases or memes shared by different users;
- Automated or bot-like posting patterns.

Importantly, no completely duplicate rows (excluding ID) were found, meaning each post has unique metadata even when content is identical.

Embedding Analysis

Along with the textual data, BERT embeddings have been provided for all posts in both the real and synthetic datasets. This section analyzes the quality and distribution of these embeddings to understand their suitability for the link prediction task.

Real vs Synthetic Embedding Comparison

The BERT embeddings for posts exhibit significantly different distributions between real and synthetic data. Table 2.4 summarizes the key statistics.

Table 2.4: BERT embedding statistics: Real vs Synthetic posts

Metric	Real	Synthetic
Number of posts	182,545	9,990
Embedding dimension	768	768
Within-group cosine similarity (mean)	0.821	0.924
Within-group cosine similarity (std)	0.097	0.019
Euclidean distance within-group (mean)	6.05	3.88
Within-group variance (mean per dim)	0.026	0.010
Cross-group cosine similarity (mean)	0.839	
Cross-group centroid distance	2.61	

The most striking difference is in the within-group cosine similarity: synthetic posts show a mean similarity of 0.924 with very low standard deviation (0.019), indicating that synthetic posts are extremely homogeneous. In contrast, real posts have a mean similarity of 0.821 with much higher variance (std=0.097).

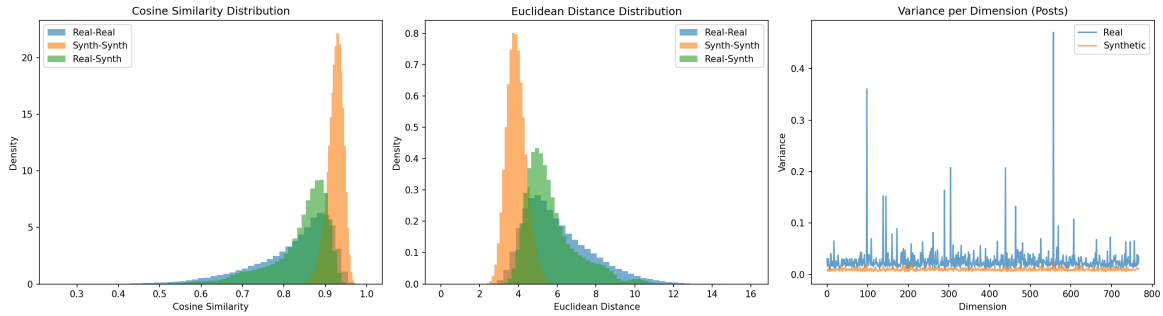


Figure 2.1: Comparison of cosine similarity distributions between real and synthetic embeddings. Synthetic posts cluster tightly with similarities concentrated around 0.90-0.95, while real posts show a broader distribution.

Embedding Space Visualization

To visualize the embedding space structure, PCA and t-SNE projections were computed for both post and user-level embeddings. Figure 2.2 shows the post-level visualization.

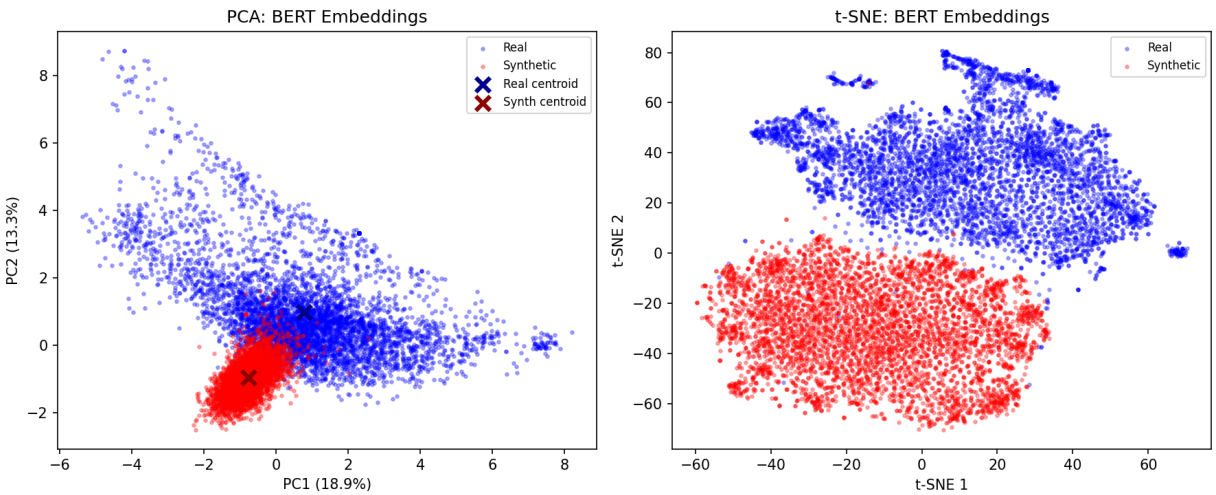


Figure 2.2: PCA and t-SNE visualization of post embeddings. The synthetic posts (red) form a tight, well-defined cluster, while real posts (blue) spread across a much larger region of the embedding space.

Figure 2.3 presents the user-level visualization, where the homogeneity of synthetic users becomes even more apparent.

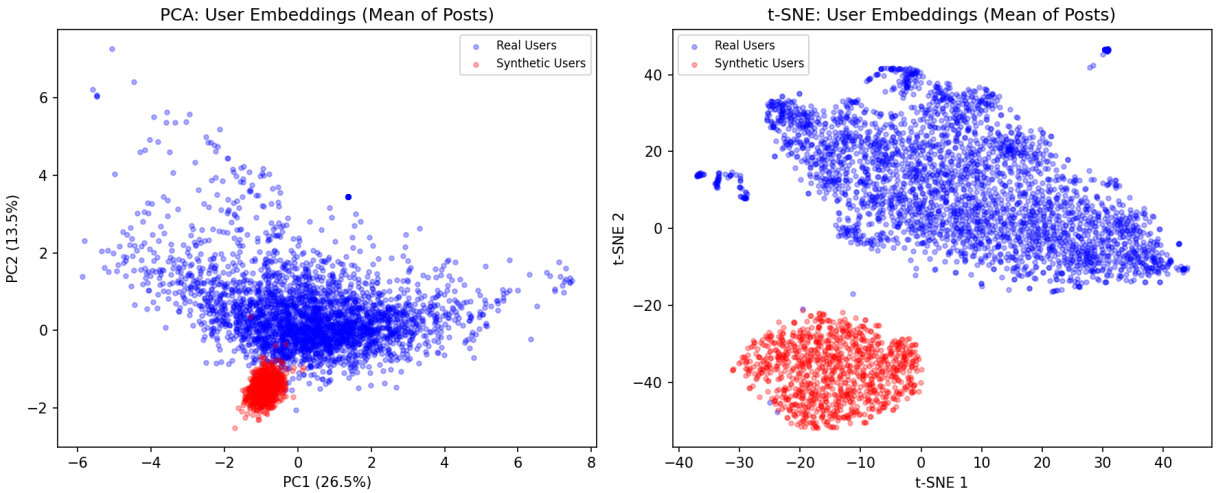


Figure 2.3: PCA and t-SNE visualization of user embeddings (mean of posts per user). Synthetic users (red) cluster in an extremely tight region, while real users (blue) occupy a much broader space. The 11.23x variance ratio is clearly visible.

Variance Analysis

The variance per embedding dimension reveals a significant difference in the representational space. For posts, the variance ratio (real/synthetic) is **2.67x**, with real posts showing a mean variance of **0.0264** per dimension compared to **0.0099** for synthetic posts.

At the user level, the disparity is even more pronounced. Table 2.5 presents the variance comparison for both posts and users.

Table 2.5: Variance comparison: Real vs Synthetic (posts and users)

Level	Real	Synthetic	Ratio
Posts within-group variance	0.0264	0.0099	2.67x
Users within-group variance	0.0174	0.0015	11.23x
Posts cross-group centroid distance		2.61	
Users cross-group centroid distance		2.57	

We distinguish between two types of variance metrics:

- **Within-group variance**: measures how much embeddings vary *within* each group. For example, real user within-group variance (0.0174) indicates how much real users differ from each other. The ratio of 11.23x means synthetic users are much more homogeneous than real users.
- **Cross-group centroid distance**: measures the Euclidean distance between the centroids (mean embeddings) of the two groups. A distance of 2.57 for users indicates that real and synthetic users occupy distinct but not extremely distant regions of the embedding space.

User-Level Embedding Aggregation

When aggregating post embeddings by user (computing the mean embedding for each user's posts), the homogeneity becomes even more pronounced. Table 2.6 presents the user-level statistics for both real and synthetic users.

Table 2.6: User-level embedding statistics: Real vs Synthetic users

Metric	Real	Synthetic
Total users	27,002	1,000
User-User cosine similarity (mean)	0.872	0.988
User-User cosine similarity (std)	0.083	0.004
Within-group variance (mean per dim)	0.0174	0.0015
Cross-group cosine similarity (mean)	0.893	

The cross-group cosine similarity of **0.893** indicates moderate semantic similarity between real and synthetic users. However, the within-group metrics reveal stark differences: synthetic users have a within-group cosine similarity of **0.988** with standard deviation of only 0.004, meaning they are nearly indistinguishable from each other. Real users, in contrast, show a mean similarity of 0.872 with much higher variance (std=0.083), reflecting natural diversity in content and interests.

3 Data Preparation

This chapter covers the data preparation phase of the CRISP-DM methodology.

3.1 Select Data

The data selection process focuses on preparing the temporal graph data for link prediction using EvolveGCN. The relevant data consists of two main components:

- **Social Network Graphs**: The edge lists from each snapshot representing the directed follower relationships (X follows Y) between users.
- **Post Content**: The textual content from posts used to derive node features through BERT embeddings.

For the incremental learning approach, we use the cumulative structure where each snapshot S_i contains all edges from previous snapshots plus new edges added in that time period. The dataset is organized into six temporal snapshots: 2016-2021, 2022, 2023, 2024, Jan-Jul 2025, and Jul 2025.

Feature Selection

Node features are derived from pre-computed BERT embeddings of user posts. For each user, the embedding is computed as the average of BERT embeddings across all their posts up to the current snapshot:

$$\mathbf{h}_u^{(t)} = \frac{1}{|P_u^{(t)}|} \sum_{p \in P_u^{(t)}} \text{BERT}(\text{content}_p) \tag{3.1}$$

where $P_u^{(t)}$ is the set of posts by user u up to snapshot t. This yields a 768-dimensional feature vector per node, capturing the semantic content of each user's contributions.

3.2 Clean Data

Missing Values

The data quality analysis revealed minimal missing values:

- The `language` attribute has 877 missing values (0.48%) in the aggregated dataset
- All other attributes are 100% complete
- No empty post content was found

Missing language values are attributed to automatic language detection failures on very short or mixed-language posts. For the link prediction task, this does not affect model performance since features are derived from BERT embeddings rather than language labels.

Duplicate Handling

Content-level duplicates were identified across snapshots, with the duplicate rate increasing over time (from 1.79% in 2016-2021 to 10.04% in Jul 2025). Two types of content duplicates exist:

- **Intra-user duplicates**: The same user posting identical content multiple times (reposts)
- **Inter-user duplicates**: Different users sharing the same post content (shared posts, quotes, or viral content)

For inter-user duplicates, the BERT embeddings computed for the identical posts are necessarily identical, since the embedding function is deterministic and content-based. This creates a semantic link between users who share the same content, which may be beneficial for link prediction as it captures content-based similarity. For link prediction, these duplicates do not negatively impact the task since:

1. The network structure is defined by edges, not post content
2. Post IDs remain unique even when content is duplicated
3. User embeddings aggregate all posts, so duplicate content reinforces rather than distorts the representation
4. Identical embeddings for shared content naturally capture content-based user similarity

3.3 Construct Data

Node Feature Construction

Temporal node features are constructed respecting the incremental nature of the dataset. For snapshot t, the features for each node are computed using only posts that exist up to and including snapshot t:

$$\mathbf{h}_u^{(t)} = \frac{1}{|P_u^{(t)}|} \sum_{p \in P_u^{(t)}} \text{BERT}(\text{content}_p) \tag{3.2}$$

where $P_u^{(t)} = \{p : \text{snapshot}(p) \leq t\}$.

This temporal feature construction ensures no data leakage: at training time for snapshot t, the model only has access to information that would have been available at that point in time.

Graph Construction

The social network is represented as a directed graph $G_t = (V_t, E_t)$ at each snapshot t, where:

- V_t is the set of nodes (users) that appear in any edge up to snapshot t (cumulative)
- E_t is the set of directed edges (follower relationships) present at snapshot t
- Edges represent "X follows Y" relationships and are treated as unweighted

The cumulative assumption states that once an edge exists, it persists in all subsequent snapshots.

3.4 Integrate Data

Data integration involves combining the edge lists with post-derived features:

1. **Edge Loading**: All .edg files are loaded and annotated with their corresponding snapshot index
2. **Post-to-User Mapping**: A mapping from post ID to user ID enables attribution of posts to nodes
3. **Post-to-Snapshot Mapping**: A mapping from post ID to snapshot index enables temporal feature filtering

The integration produces a unified dataset object containing:

- Edge tensor with shape $[num_edges, 3]$ (source, target, timestep)
- Node count and per-snapshot node statistics
- Feature dimension (768 for BERT)
- ID mappings for interpretation

3.5 Format Data

Adjacency Matrix Format

For the GCN operations, adjacency matrices are stored in sparse format:

$$\hat{A}_t = \tilde{D}_t^{-\frac{1}{2}} \tilde{A}_t \tilde{D}_t^{-\frac{1}{2}}, \quad \tilde{A}_t = A_t + I \tag{3.3}$$

where the normalization enables stable gradient flow through the graph convolution layers.

The sparse representation uses:

- idx: $[2, num_edges]$ tensor of (source, target) pairs
- vals: $[num_edges]$ tensor of edge values (normalized)

The number of edges depends on the snapshot considered at the time of training or inference, following the cumulative structure.

Training Sample Format

What the model will receive during training for snapshot t is:

- **Historical Adjacency List**: $[\hat{A}_{t-k}, \dots, \hat{A}_{t-1}]$ for k historical steps
- **Historical Node Features**: $[H_{t-k}, \dots, H_{t-1}]$ corresponding feature matrices
- **Node Masks**: Binary masks indicating which nodes are active at each historical step
- **Label Edges**: Pairs (u, v) with labels $y \in \{0, 1\}$ for training/evaluation

4 Modeling

This chapter covers the modeling phase of the CRISP-DM methodology.

4.1 Select Modeling Techniques

For dynamic graph analysis, we selected **EvolveGCN** (Evolving Graph Convolutional Networks) [8] as our primary modeling approach. This technique addresses a critical limitation of existing methods: the requirement for nodes to be present throughout the entire time span (training and testing), making them less applicable when node sets frequently change or completely differ between time steps.

Motivation and Context

Traditional graph neural network approaches for dynamic graphs typically resort to node embeddings and use recurrent neural networks to regulate these embeddings and learn temporal dynamics. However, this approach faces significant challenges in practical scenarios where:

- New nodes emerge after training
- Nodes frequently appear and disappear
- Node sets at different time steps may completely differ

EvolveGCN resolves these challenges by focusing on model adaptation rather than node embeddings. Instead of using RNNs to learn dynamics from node features extracted by GNNs, EvolveGCN uses RNNs to regulate the GCN model (network parameters) at every time step [8].

Architecture Overview

EvolveGCN adapts the Graph Convolutional Network (GCN) model along the temporal dimension without resorting to node embeddings. The proposed approach captures the dynamism of the graph sequence through using an RNN to evolve the GCN parameters, with two architectures considered for parameter evolution [8].

At each time step t, the input data consists of the pair $(A_t \in \mathbb{R}^{n \times n}, X_t \in \mathbb{R}^{n \times d})$, where A_t is the graph adjacency matrix and X_t is the matrix of node features (each row is a d-dimensional feature vector).

Graph Convolution Layer

A GCN consists of multiple layers of graph convolution, which is similar to a perceptron but additionally has a neighborhood aggregation step motivated by spectral convolution. At time t, the l-th layer takes the adjacency matrix A_t and the node embedding matrix $H_t^{(l)}$ as input, and uses a weight matrix $W_t^{(l)}$ to update the node embedding matrix to $H_t^{(l+1)}$ as output:

$$H_t^{(l+1)} = \mathrm{GCONV}(A_t, H_t^{(l)}, W_t^{(l)}) := \sigma(\hat{A}_t H_t^{(l)} W_t^{(l)}) \tag{4.1}$$

where $\hat{A}_t$ is a normalization of A_t defined as:

$$\hat{A} = \tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}, \quad \tilde{A} = A + I, \quad \tilde{D} = \text{diag}\left(\sum_j \tilde{A}_{ij}\right) \tag{4.2}$$

and σ is the activation function (typically ReLU) for all but the output layer.

Weight Evolution Mechanisms

EvolveGCN-H (Hidden State Version). The first option treats $W_t^{(l)}$ as the hidden state of the dynamical system. A Gated Recurrent Unit (GRU) is used to update the hidden state upon time-t input to the system, where the input information is naturally the node embeddings $H_t^{(l)}$:

$$W_t^{(l)} = \text{GRU}(H_t^{(l)}, W_{t-1}^{(l)}) \tag{4.3}$$

The implementation requires to match the column dimension of the input with that of the hidden state. This is achieved through a **summarization function** that condenses the node embedding matrix $H_t^{(l)}$ into a matrix with appropriate dimensions using a learned parameter vector to compute weights for the rows, selecting the top-k weighted rows.

EvolveGCN-O (Output Version). The second option treats $W_t^{(l)}$ as the output of the dynamical system (which becomes the input at the subsequent time step). A Long Short-Term Memory (LSTM) cell models this input-output relationship, maintaining system information through a cell context that acts like the hidden state of a GRU:

$$W_t^{(l)} = \text{LSTM}(W_{t-1}^{(l)}) \tag{4.4}$$

In this version, node embeddings are not used at all.

Evolving Graph Convolution Unit (EGCU)

Combining the graph convolution unit GCONV and a recurrent architecture, we reach the Evolving Graph Convolution Unit (EGCU). Depending on the way GCN weights are evolved, we have two versions:

For EvolveGCN-H:

$$W_t^{(l)} = \text{GRU}(H_t^{(l)}, W_{t-1}^{(l)}) \tag{4.5}$$
$$H_t^{(l+1)} = \text{GCONV}(A_t, H_t^{(l)}, W_t^{(l)}) \tag{4.6}$$

For EvolveGCN-O:

$$W_t^{(l)} = \text{LSTM}(W_{t-1}^{(l)}) \tag{4.7}$$
$$H_t^{(l+1)} = \text{GCONV}(A_t, H_t^{(l)}, W_t^{(l)}) \tag{4.8}$$

In the -H version, the GCN weights are treated as hidden states of the recurrent architecture; in the -O version, these weights are treated as input/outputs. Chaining the units bottom-up creates a GCN with multiple layers for one time step.

Chosen Configuration

We chose to use the EvolveGCN-H variant for our experiments, as it allows the GCN weights to be directly influenced by the node embeddings at each time step.

GCN Architecture Details The GCN component consists of two stacked Graph Recurrent Convolution Units (GRCUs), each implementing a single graph convolution layer with weight evolution. The architecture follows a progressive dimensionality reduction pattern:

- **Layer 1**: Input dimension 768 (BERT embedding size) → Output dimension 128
- **Layer 2**: Input dimension 128 → Output dimension 64

Each GRCU layer performs the following operations at each time step t:

1. **Weight Evolution**: The GCN weight matrix $W_t^{(l)}$ is evolved using a matrix-based GRU cell that takes as input the previous weights and the current node embeddings:

$$W_t^{(l)} = \text{matGRU}(W_{t-1}^{(l)}, H_t^{(l)}, \text{mask}) \tag{4.9}$$

2. **Top-K Selection**: A learnable scoring mechanism selects the k most relevant node embeddings for weight evolution. The scorer computes attention weights as:

$$\text{scores} = \frac{H_t^{(l)} \cdot s}{\|s\|}, \quad s \in \mathbb{R}^{d \times 1} \tag{4.10}$$

   where s is a learnable parameter vector. The top-k nodes are selected and their embeddings are weighted using tanh(scores).

3. **Graph Convolution**: The evolved weights are applied to perform neighborhood aggregation:

$$H_t^{(l+1)} = \sigma(\hat{A}_t \cdot H_t^{(l)} \cdot W_t^{(l)}) \tag{4.11}$$

   where $\hat{A}_t$ is the normalized adjacency matrix and σ is the ReLU activation function.

The matrix GRU cell implements three gates (update, reset, and candidate) following the standard GRU formulation, but extended to operate on matrices rather than vectors. Each gate is defined as:

$$g = \phi(W_g \cdot z_{\text{topk}} + U_g \cdot W_{\text{prev}} + b_g) \tag{4.12}$$

where ϕ is either sigmoid (for update and reset gates) or tanh (for candidate gate), and z_{topk} is the output of the Top-K selector.

MLP Classifier The node embeddings produced by the final GCN layer are fed into a two-layer Multi-Layer Perceptron (MLP), concatenating pairs of node embeddings to predict the existence of edges between them.

The MLP architecture is:

- **Input layer**: Dimension 128 (64×2 from concatenated node embeddings)
- **Activation**: ReLU
- **Output layer**: 2 logits (edge exists / edge does not exist)

The classifier outputs probability scores for edge existence, trained with binary cross-entropy loss using balanced class weights $[0.5, 0.5]$ to handle the imbalance between positive and negative edge samples:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[w_1 y_i \log(\hat{y}_i) + w_0(1-y_i)\log(1-\hat{y}_i)\right] \tag{4.13}$$

Experiment Design: Training Protocol

In this project, we implemented two distinct training pipelines using the H variant of the EvolveGCN architecture:

- **Original Implementation**: Sequence-based training where all snapshots are processed in a single batch per epoch
- **Incremental Implementation**: Phase-based training where each snapshot is processed in a dedicated training phase

Original Implementation

This pipeline follows the reference implementation of EvolveGCN.
Given n training snapshots $\{S_0, S_1, \dots, S_{n-1}\}$:

- **Batch Composition**: Each training batch contains all n snapshots. The model processes the entire temporal sequence in a single forward pass.
- **Temporal Processing**: At each step t within the batch, the snapshot S_t observes the history accumulated up to S_{t-1}. The EvolveGCN architecture internally manages weight evolution across time steps through its recurrent mechanism (GRU).
- **Training Loop**: For E epochs, the model repeatedly sees the same sequence of snapshots, learning to predict edges at each time step based on historical context.

The best final model, chosen according to the MAP metric on the last training snapshot, is used for inference on synthetic data.

Incremental Implementation

Given n training snapshots, the training is divided into n distinct phases. Each phase i is dedicated to a single snapshot S_i:

- **Phase Isolation**: During Phase i, the model trains for E epochs exclusively on snapshot S_i. The snapshot S_i contains the graph accumulated up to time i, including all edges from previous snapshots.
- **Weight Transfer**: At the end of Phase i, the best checkpoint is saved. Phase $i+1$ starts by loading these weights.
- **Evaluation**: After training on S_i, the model is evaluated on S_{i+1}.

For example, with 4 training snapshots:

1. **Phase 0**: Train for E epochs on S_0
2. **Phase 1**: Load best weights from Phase 0, train for E epochs on S_1

3. **Phase 2**: Load best weights from Phase 1, train for E epochs on S_2
4. **Phase 3**: Load best weights from Phase 2, train for E epochs on S_3

This results in $4 \times E$ total epochs, with the model progressively adapting to each temporal snapshot. In our case, we have 5 training snapshot (= 5 phases).

The final model checkpoint from Phase 4 is used for inference on synthetic data.

Experimental Configurations

We designed four experimental configurations per pipeline to evaluate the impact of two key hyper-parameters: **learning rate** and **negative sampling ratio**. In particular, we tested learning rates of 0.001 and 0.0005, and negative sampling multipliers of 1× and 2× (i.e., generating one or two negative samples per positive edge).

For the **Original Implementation**, 50 epochs are used to train on the entire sequence of snapshots in a single batch per epoch. For the **Incremental Implementation**, 50 epochs are used per phase, accumulating to 250 total epochs across 5 phases. All four configurations share the same architecture described in Section 4.1.

A more aggressive negative sampling strategy is not explored due to the significant dimension of the dataset and the computational cost of training.

4.2 Generate Test Design

As **classification metrics**, we will use:

- **MAP (Mean Average Precision):** Mean of per-query average precisions:

$$\text{MAP} = \frac{1}{Q} \sum_{q=1}^{Q} \text{AP}(q),$$

  where

$$\text{AP}(q) = \frac{1}{m} \sum_{k=1}^{m} \text{Precision@}k$$

  over relevant items.

- **Macro Precision:** Unweighted mean of per-class precision:

$$\frac{1}{C} \sum_{c=1}^{C} \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}.$$

- **Macro Recall:** Unweighted mean of per-class recall:

$$\frac{1}{C} \sum_{c=1}^{C} \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}.$$

- **Macro F1:** Unweighted mean of per-class F1 scores:

$$\frac{1}{C} \sum_{c=1}^{C} \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}.$$

- **AUC-ROC:** Illustrates the performance of a binary classifier system as its discrimination threshold is varied. It is created by plotting the fraction of true positives out of the positives (TPR = true positive rate) vs. the fraction of false positives out of the negatives (FPR = false positive rate), at various threshold settings.

- **Per-class metrics:** Precision, Recall and F1 reported individually for each class to inspect class-level behavior.

To **evaluate the graph structure**, we will use:

- **Average Degree:** the mean number of connections per node. For undirected graphs

$$\text{AvgDeg} = \frac{2|E|}{|V|},$$

  while for directed graphs we use

$$\text{AvgDeg} = \frac{|E|}{|V|},$$

  where $|E|$ and $|V|$ denote the number of edges and vertices, respectively.

- **Average Clustering Coefficient:** measures the extent to which neighboring nodes tend to connect to each other, quantifying local network transitivity. The local clustering coefficient for node i is

$$C_i = \frac{2e_i}{k_i(k_i - 1)},$$

  where e_i is the number of edges between neighbors of node i and k_i is the degree of node i. The average clustering coefficient is the mean across all nodes:

$$\text{AvgCC} = \frac{1}{|V|} \sum_{i \in V} C_i.$$

- **Average Shortest Path:** estimates the typical distance between nodes, reflecting how far information must travel through the network. For each pair of nodes (i, j), we compute the shortest path length $d(i, j)$ as the minimum number of edges required to reach from i to j. The average is computed over all reachable pairs within the largest connected component (LCC):

$$\text{AvgSP} = \frac{1}{|P|} \sum_{(i,j) \in P} d(i, j),$$

  where P is the set of all reachable node pairs and $|P|$ is their count. Lower values indicate more efficient information dissemination. For directed graphs we compute shortest paths considering the weakly connected component. Networks with AvgSP < 5 are typically considered well-connected small-world networks.

- **Modularity:** a measure of community structure quality, quantifying the strength of division into modules compared to random expectations. It typically ranges from -1 to 1, with high positive values (e.g., > 0.3) indicating strong community structure. The modularity is computed as

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j),$$

  where m is the total number of edges, k_i is the degree of node i, A_{ij} is the adjacency matrix entry,

and $\delta(c_i, c_j)$ equals 1 if nodes i and j are in the same community, 0 otherwise. For directed graphs we convert to undirected representation before the computation.

We will compare the final cumulative graph provided by the dataset with the synthetic graph generated by the models, along with other social network datasets [9], specifically:

- **Facebook Social Circles:**[1] A dataset of ego-networks from Facebook, containing 4039 nodes and 88234 edges.
- **Twitter Social Circles:**[2] A dataset of ego-networks from Twitter, containing 81306 nodes and 1768149 edges.
- **Google+ Social Circles:**[3] A dataset of ego-networks from Google+, containing 107614 nodes and 13673453 edges.

We want to analyze the structural properties of the generated graph on the synthetic dataset. The ideal result will be a network that resembles the real cumulative graph in terms of degree distribution, clustering coefficient, average path length, and modularity. The other datasets will give context on whether eventual different metrics are still valuable and encountered on other real social networks.

To generate the synthetic network, the pipeline will be as follows:

1. **Model Loading**: Load the trained EvolveGCN-H model from the final checkpoint (phase 4) of the incremental training pipeline. The model includes both the GCN component (with evolved weights) and the MLP classifier for edge prediction.
2. **Real Graph Construction**: Load the real Gab network up to the final snapshot, including:
   - Adjacency matrix with all historical edges
   - Node features (BERT embeddings) for all 25,569 real users
3. **Synthetic Data Loading**: Load pre-computed BERT embeddings for synthetic users. Synthetic users are distributed across 3 batches (400, 400, and 200 users, totaling 1,000 synthetic users).
4. **Incremental Batch Processing**: For each batch of synthetic users:
   (a) *User Embedding Computation*: Compute each synthetic user's embedding as the average of their post embeddings.
   (b) *Hybrid Graph Construction*: Build an adjacency matrix combining:
      - Real edges from the Gab network
      - Self-loops for all synthetic nodes (to ensure GCN can process isolated nodes)
   (c) *Feature Matrix Assembly*: Concatenate real node features with cumulative synthetic user embeddings.
   (d) *GCN Embedding*: Compute node embeddings for all nodes (real + synthetic) using the trained GCN.
   (e) *Edge Prediction*: Predict probabilities for candidate edges in three categories:
      - New synthetic → all nodes (real, old synthetic, new synthetic)

- Real → new synthetic
- Old synthetic → new synthetic

(f) *Thresholding*: Classify edges as positive if prediction probability ≥ 0.5.

5 Evaluation

This chapter covers the evaluation phase of the CRISP-DM methodology, assessing whether the developed models meet the business success criteria defined in Section 1.1.

5.1 Assess Model

The experimental configurations reported in the previous chapter were evaluated on the Jul 2025 snapshot. Table 5.1 presents the results, while Table 5.2 shows per-class metrics.

Table 5.1: Test metrics comparison: Incremental vs. Original implementation

	LR	NM	MAP	AUC	Mac P	Mac R	Mac F1
Incr.	0.001	1	0.78	0.78	0.25	0.49	0.33
	0.001	2	0.64	0.74	0.75	0.68	0.69
	0.0005	1	0.78	0.80	0.68	0.52	0.38
	0.0005	2	0.62	0.72	0.73	0.68	0.69
Orig.	0.001	1	0.87	0.63	0.86	0.86	0.86
	0.001	2	0.77	0.65	0.82	0.83	0.83
	0.0005	1	0.86	0.64	0.82	0.79	0.78
	0.0005	2	0.76	0.65	0.81	0.84	0.81

Incr. = Incremental, Orig. = Original, LR = Learning Rate, NM = Negative Multiplier, Mac P = Macro Precision, Mac R = Macro Recall

Table 5.2: Per-class metrics comparison: Incremental vs. Original implementation

	LR	NM	Class 0 (Non-link)			Class 1 (Link)		
			P	R	F1	P	R	F1
Incr.	0.001	1	0.00	0.00	0.00	0.49	0.97	0.65
	0.001	2	0.77	0.92	0.83	0.73	0.44	0.55
	0.0005	1	0.85	0.04	0.08	0.51	0.99	0.67
	0.0005	2	0.77	0.90	0.83	0.69	0.47	0.56
Orig.	0.001	1	0.89	0.83	0.86	0.84	0.89	0.87
	0.001	2	0.89	0.88	0.88	0.76	0.78	0.77
	0.0005	1	0.93	0.62	0.74	0.71	0.96	0.82
	0.0005	2	0.95	0.76	0.84	0.66	0.93	0.77

Incr. = Incremental, Orig. = Original, LR = Learning Rate, NM = Negative Multiplier, P = Precision, R = Recall

Overall Performance Comparison. The Original implementation achieves significantly higher MAP (0.87 vs. 0.78) and Macro F1 (0.86 vs. 0.69), indicating superior ranking quality and classification performance. However, the Incremental approach achieves notably higher AUC (0.80 vs. 0.65), suggesting better discrimination ability at various probability thresholds.

Choosing MAP as discriminative metric. Choosing MAP as the primary metric for model selection is justified by its focus on ranking quality, which is critical for link prediction tasks. However, as we can see in the first configuration of the Incremental approach, the best model predicts always 1, in practice the model is not able to discriminate between the two classes.

Negative Sampling Multiplier Impact. For both approaches, increasing NM from 1 to 2, it reduces MAP (Incremental: 0.78 → 0.64; Original: 0.87 → 0.77), as expected. However, it improves AUC (Incremental: 0.78 → 0.72; Original: 0.63 → 0.65), suggesting that a higher proportion of negative samples helps the model learn better decision boundaries, albeit at the cost of ranking quality.

Learning Rate Impact. The learning rate shows different effects across approaches:

- **Incremental**: LR=0.0005 achieves higher AUC (0.8037) but with worse class balance
- **Original**: LR=0.001 consistently outperforms LR=0.0005 across all metrics

In general, the original approach demonstrates stronger performance in terms of MAP and Macro F1.

Graph Metrics Analysis

Table 5.3 presents the comparison across degree, modularity, average path length, and clustering coefficient. We report only the metrics computed on graph where it's worth to: infact, some configuration tends to always assign the same label to all the edges, so the resulting graph is either a complete graph or an empty graph, and in both cases the metrics are not meaningful. For the first case (complete graph), this can be due to the training process not converging properly, while for the second case (empty graph) this can be due to a lack of generalization of the model, or from the nature of the real and synthetic data that, as analyzed in the previous chapters, are clustered in different part of the space.

Table 5.3: Graph metrics comparison: Real vs. Synthetic Networks

Network	Avg Degree	Modularity	Avg Path	Clustering Coeff.
Facebook	43.69	0.8348	3.6167	0.6055
Twitter	21.75	0.8042	4.9044	0.5653
Google+[4]	127.06	0.4742	3.3163	0.4901
Gab (Ours)	32.11	0.2742	3.0384	0.0808
Incr., LR=0.0005, NM=1	220.35	-0.0013	1.0167	0.5941

Analysis of Synthetic Networks

The synthetic network generated by the Incremental configuration (LR=0.0005, NM=1) exhibits significant structural differences compared to real social networks:

Degree Distribution. The synthetic network shows an extremely high average degree (220.35), approximately 7× higher than the Gab dataset (32.11) and 5× higher than Google+ (127.06). This indicates that the model tends to predict edges too liberally, creating an overly dense network.

Community Structure. The near-zero modularity (-0.0013) indicates a complete absence of community structure. Real social networks typically exhibit modularity values between 0.3 and 0.8, reflecting the natural tendency of users to form groups.

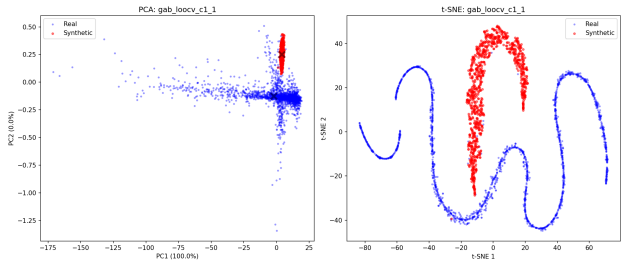
[4]Due to the dimension of the Google+ dataset, we report the metrics for the largest connected component (LCC) considering the 20% of nodes with the highest degree.

Small-World Property. The average path length of 1.0167 is unrealistically low compared to real networks (3.0-4.9). This is a direct consequence of the high density: in a nearly complete graph, most nodes are directly connected or separated by only one intermediate node. Real social networks maintain the "small-world" property with moderate path lengths (typically 3-6) while remaining sparse.

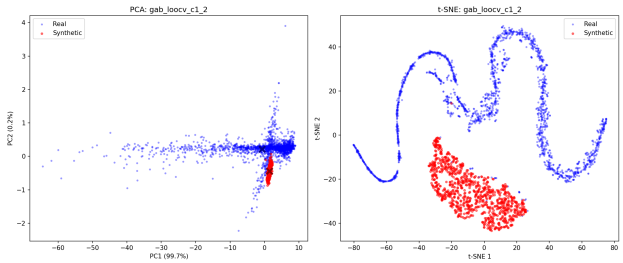
Clustering Coefficient. Interestingly, the clustering coefficient (0.5941) falls within the range of real social networks (0.49-0.61). This suggests that when the model does predict edges, it tends to close triangles. However, this metric is somewhat misleading given the extreme density of the network.

GCN Embedding Visualization

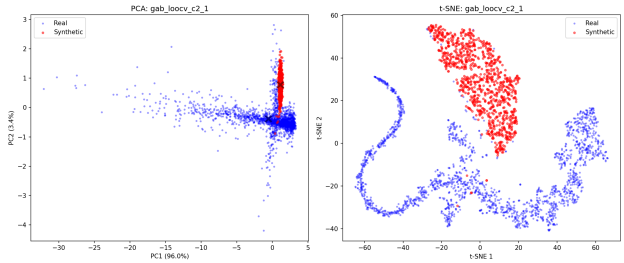
To better understand the behaviors of our model, we visualized the GCN embeddings using PCA and t-SNE dimensionality reduction. Figures 5.1 and 5.2 present the embeddings for the Original and Incremental configurations, showing the distribution of real users (blue) and synthetic users (red) in the learned embedding space.



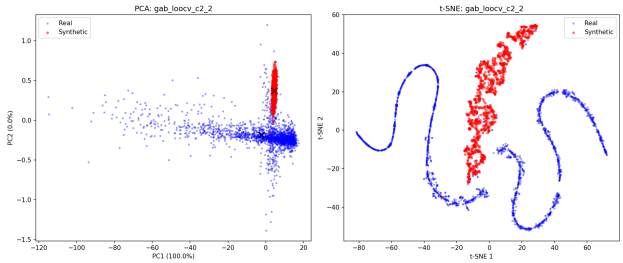
(a) Config 1, NM=1 (LR=0.001)



(b) Config 1, NM=2 (LR=0.001)

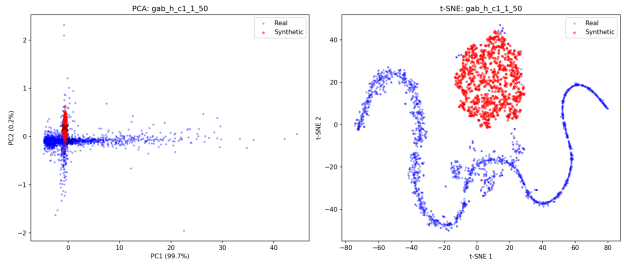


(c) Config 2, NM=1 (LR=0.0005)

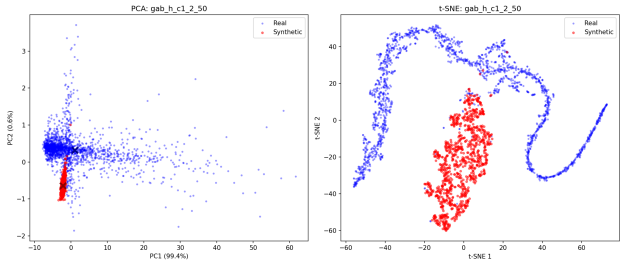


(d) Config 2, NM=2 (LR=0.0005)

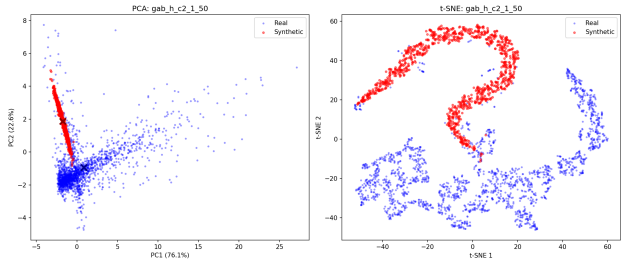
Figure 5.1: PCA and t-SNE visualization of GCN embeddings for Original implementation. Blue points represent real users, red points represent synthetic users.



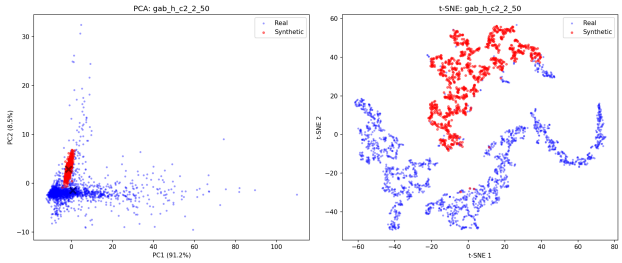
(a) Config 1, NM=1 (LR=0.001)



(b) Config 1, NM=2 (LR=0.001)



(c) Config 2, NM=1 (LR=0.0005)



(d) Config 2, NM=2 (LR=0.0005)

Figure 5.2: PCA and t-SNE visualization of GCN embeddings for Incremental implementation. Blue points represent real users, red points represent synthetic users.

Table 5.4 summarizes the PCA statistics for each configuration, including the explained variance ratio and the centroid distance between real and synthetic user embeddings.

Table 5.4: PCA statistics for GCN embeddings

	Config	PC1 Explained	PC2 Explained	Total Explained	Centroid Distance
Orig.	Config 1, NM=1	99.98%	0.01%	100.00%	6.29
	Config 1, NM=2	99.71%	0.23%	99.94%	2.33
	Config 2, NM=1	95.99%	3.39%	99.38%	2.06
	Config 2, NM=2	99.96%	0.03%	100.00%	6.65
Incr.	Config 1, NM=1	99.65%	0.21%	99.86%	1.01
	Config 1, NM=2	99.39%	0.57%	99.96%	3.41
	Config 2, NM=1	76.13%	22.61%	98.74%	3.84
	Config 2, NM=2	91.20%	8.47%	99.67%	4.78

The analysis reveals several key observations:

Dimensional Collapse and Oversmoothing. The Original implementation configurations show extreme concentration in the first principal component (95-100%), indicating **dimensional collapse**: the GCN embeddings collapse to a nearly one-dimensional space. This phenomenon is closely related to **oversmoothing** in Graph Neural Networks, where repeated message passing operations cause node representations to become indistinguishable from one another [10, 11]. After multiple GCN layers, each node's embedding becomes dominated by the aggregated neighborhood statistics rather than its individual features.

In contrast, the Incremental implementation shows more variance distribution: Config 2 with NM=1 achieves only 76.13% on PC1, with 22.61% on PC2, suggesting a richer embedding space with less severe oversmoothing. This may explain why that particular run is the only one that attempts to connect synthetic users, even if with poor performance.

Centroid Distance. The centroid distance between real and synthetic embeddings correlates strongly with classification performance:

- **Original Config 1 NM=1 (6.29)** and **Config 2 NM=2 (6.65)**: Excellent separation, aligning with the highest MAP scores (0.8660) and balanced F1.
- **Incremental Config 1 NM=1 (1.01)**: Poor separation, correlated to the fact that the model predicts exclusively Class 1.
- **Incremental Config 2 NM=1 (3.84)** and **Config 2 NM=2 (4.78)**: Moderate separation with richer embedding structure (lower PC1 explained variance).

We can say that the failure in connecting synthetic users for the original configurations is due to the fact that the model learns a very narrow embedding space where real and synthetic users are perfectly separated, but the synthetic users are clustered in a single point, so the model is not able to learn any discriminative feature for them, and it ends up predicting always 0 for the edges between synthetic users. On the other hand, the incremental approach that link synthetic users is the one with the lowest explained variance on PC1, and the highest explained variance on PC2, so the model learns a richer embedding space where real and synthetic users are not perfectly separated, but the synthetic users are spread across the space, so the model is able to learn some discriminative features for them, even if with poor performance.

Effect of Negative Sampling. Increasing NM from 1 to 2 generally improves centroid distance separation, particularly for the Incremental approach (1.01 → 3.41 for Config 1). This confirms that higher negative sampling encourages the model to learn more discriminative features.

Bibliography

[1] Shazia Tabassum, Fabiola S. F. Pereira, Sofia Fernandes, and Jo
ao Gama. Social network analysis: An overview. *WIREs Data Mining and Knowledge Discovery*, 8(5):e1256, September 2018. 1, 2

[2] Nur Nasuha Daud, Siti Hafizah Ab Hamid, Muntadher Saadoon, Firdaus Sahran, and Nor Badrul Anuar. Applications of link prediction in social networks: A review. *Journal of Network and Computer Applications*, 166:102716, September 2020. 1, 2

[3] Antonio Pellicani, Gianvito Pio, Domenico Redavid, and Michelangelo Ceci. SAIRUS: Spatially-aware identification of risky users in social networks. *Information Fusion*, 92:435–449, April 2023. 1

[4] Zafran Khan, Zeeshan Khan, Byung-Geun Lee, Hong Kook Kim, and Moongu Jeon. Graph neural networks based framework to analyze social media platforms for malicious user detection. *Applied Soft Computing*, 155:111416, April 2024. 1

[5] Francesco Benedetti, Antonio Pellicani, Gianvito Pio, and Michelangelo Ceci. Multi-view Learning for the Identification of Risky Users in Dynamic Social Networks, March 2025. 1

[6] Francesco Benedetti, Antonio Pellicani, Gianvito Pio, and Michelangelo Ceci. IMMENSE: Inductive Multi-perspective User Classification in Social Networks. *Online Social Networks and Media*, 50:100335, December 2025. 1, 2

[7] Roberto Corizzo, Gianvito Pio, Emanuele Pio Barracchia, Antonio Pellicani, Nathalie Japkowicz, and Michelangelo Ceci. HURI: Hybrid user risk identification in social networks. *World Wide Web*, 26(5):3409–3439, September 2023. 1

[8] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao Schardl, and Charles Leiserson. EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. 34(04):5363–5370. 13

[9] Jure Leskovec and Julian Mcauley. Learning to discover social circles in ego networks. *Advances in neural information processing systems*, 25, 2012. 19

[10] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. 24

[11] Kenta Oono and Taiji Suzuki. Graph neural networks exponentially lose expressive power for node classification. *International Conference on Learning Representations*, 2020. 24